

# 6. Estructuras de datos

Este capítulo describe algunas cosas que ya aprendiste en más detalle, y agrega algunas cosas nuevas también.

## 6.1. Más sobre listas

El tipo de dato lista tiene algunos métodos más. Aquí están todos los métodos de los objetos lista:

### **list.append(x)**

Agrega un ítem al final de la lista. Equivale a `a[len(a):] = [x]`.

### **list.extend(iterable)**

Extiende la lista agregándole todos los ítems del iterable. Equivale a `a[len(a):] = iterable`.

### **list.insert(i, x)**

Inserta un ítem en una posición dada. El primer argumento es el índice del ítem delante del cual se insertará, por lo tanto `a.insert(0, x)` inserta al principio de la lista, y `a.insert(len(a), x)` equivale a `a.append(x)`.

### **list.remove(x)**

Quita el primer ítem de la lista cuyo valor sea `x`. Es un error si no existe tal ítem.

### **list.pop([i])**

Quita el ítem en la posición dada de la lista, y lo devuelve. Si no se especifica un índice, `a.pop()` quita y devuelve el último ítem de la lista. (Los corchetes que encierran a `i` en la firma del método denotan que el parámetro es opcional, no que deberías escribir corchetes en esa posición. Verás esta notación con frecuencia en la Referencia de la Biblioteca de Python.)

### **list.clear()**

Quita todos los elementos de la lista. Equivalente a `del a[:]`.

### **list.index(x[, start[, end]])**

Devuelve un índice basado en cero en la lista del primer ítem cuyo valor sea `x`. Levanta una excepción **ValueError** si no existe tal ítem.

Los argumentos opcionales `start` y `end` son interpretados como la notación de rebanadas y se usan para limitar la búsqueda a una subsecuencia particular de la lista. El índice retornado se calcula de manera relativa al inicio de la secuencia completa en lugar de con respecto al argumento `start`.

### **list.count(x)**

Devuelve el número de veces que `x` aparece en la lista.

### **list.sort(key=None, reverse=False)**

Ordena los ítems de la lista in situ (los argumentos pueden ser usados para personalizar el orden de la lista, ve **sorted()** para su explicación).

## **list.reverse()**

Invierte los elementos de la lista in situ.

## **list.copy()**

Devuelve una copia superficial de la lista. Equivalente a `a[:]`.

Un ejemplo que usa la mayoría de los métodos de lista:

```
>>> frutas = ['naranja', 'manzana', 'pera', 'banana', 'kiwi', 'manzana', 'banana']
>>> frutas.count('manzana')
2
>>> frutas.count('mandarina')
0
>>> frutas.index('banana')
3
>>> frutas.index('banana', 4) # Find next banana starting a position 4
6
>>> frutas.reverse()
>>> frutas
['banana', 'manzana', 'kiwi', 'banana', 'pera', 'manzana', 'naranja']
>>> frutas.append('uva')
>>> frutas
['banana', 'manzana', 'kiwi', 'banana', 'pera', 'manzana', 'naranja', 'uva']
>>> frutas.sort()
>>> frutas
['manzana', 'manzana', 'banana', 'banana', 'uva', 'kiwi', 'naranja', 'pera']
>>> frutas.pop()
'pera'
```

Quizás hayas notado que métodos como `insert`, `remove` o `sort`, que solo modifican a la lista, no tienen impreso un valor de retorno – devuelven `None`.<sup>[1]</sup> Esto es un principio de diseño para todas las estructuras de datos mutables en Python.

## 6.1.1. Usando listas como pilas

Los métodos de lista hacen que resulte muy fácil usar una lista como una pila, donde el último elemento añadido es el primer elemento retirado (“último en entrar, primero en salir”). Para agregar un ítem a la cima de la pila, use **`append()`**. Para retirar un ítem de la cima de la pila, use **`pop()`** sin un índice explícito. Por ejemplo:

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```

## 6.1.2. Usando listas como colas

También es posible usar una lista como una cola, donde el primer elemento añadido es el primer elemento retirado (“primero en entrar, primero en salir”); sin embargo, las listas no son eficientes para este propósito. Agregar y sacar del final de la lista es rápido, pero insertar o sacar del comienzo de una lista es lento (porque todos los otros elementos tienen que ser desplazados por uno).

Para implementar una cola, usá **collections.deque** el cual fue diseñado para agregar y sacar de ambas puntas de forma rápida. Por ejemplo:

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # llega Terry
>>> queue.append("Graham")         # llega Graham
>>> queue.popleft()                # el primero en llegar ahora se va
'Eric'
>>> queue.popleft()                # el segundo en llegar ahora se va
'John'
>>> queue                          # el resto de la cola en orden de llegada
['Michael', 'Terry', 'Graham']
```

## 6.1.3. Comprensión de listas

Las comprensiones de listas ofrecen una manera concisa de crear listas. Sus usos comunes son para hacer nuevas listas donde cada elemento es el resultado de algunas operaciones aplicadas a cada miembro de otra secuencia o iterable, o para crear una subsecuencia de esos elementos para satisfacer una condición determinada.

Por ejemplo, asumamos que queremos crear una lista de cuadrados, como:

```
>>> cuadrados = []
>>> for x in range(10):
...     cuadrados.append(x**2)
...
>>> cuadrados
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Nota que esto crea (o sobreescribe) una variable llamada `x` que sigue existiendo luego de que el bucle haya terminado. Podemos calcular la lista de cuadrados sin ningún efecto secundario haciendo:

```
cuadrados = list(map(lambda x: x**2, range(10)))
```

o, un equivalente:

```
cuadrados = [x ** 2 for x in range(10)]
```

que es más conciso y legible.

Una lista de comprensión consiste de corchetes rodeando una expresión seguida de la declaración **for** (reference.html#for) y luego cero o más declaraciones **for** (reference.html#for) o **if** (reference.html#if). El resultado será una nueva lista que sale de evaluar la expresión en el contexto de los **for** (reference.html#for) o **if** (reference.html#if) que le siguen. Por ejemplo, esta lista de comprensión combina los elementos de dos listas si no son iguales:

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

y es equivalente a:

```
>>> combs = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
...             combs.append((x, y))
...
>>> combs
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Notá como el orden de los **for** (reference.html#for) y **if** (reference.html#if) es el mismo en ambos pedacitos de código.

Si la expresión es una tupla (como el (x, y) en el ejemplo anterior), debe estar entre paréntesis.

```

>>> vec = [-4, -2, 0, 2, 4]
>>> # crear una nueva lista con los valores duplicados
>>> [x * 2 for x in vec]
[-8, -4, 0, 4, 8]
>>> # filtrar la lista para excluir números negativos
>>> [x for x in vec if x >= 0]
[0, 2, 4]
>>> # aplica una función a todos los elementos
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> # llama un método a cada elemento
>>> fruta fresca = [' banana', ' mora de Logan ', 'maracuya ']
>>> [arma.strip() for arma in fruta fresca]
['banana', 'mora de Logan', 'maracuya']
>>> # crea una lista de tuplas de dos como (número, cuadrado)
>>> [(x, x ** 2) for x in range(6)]
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
>>> # la tupla debe estar entre paréntesis, sino es un error
>>> [x, x ** 2 for x in range(6)]
Traceback (most recent call last):
...
      [x, x ** 2 for x in range(6)]
            ^
SyntaxError: invalid syntax
>>> # aplanar una lista usando comprensión de listas con dos 'for'
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Las comprensiones de listas pueden contener expresiones complejas y funciones anidadas:

```

>>> from math import pi
>>> [str(round(pi, i)) for i in range(1, 6)]
['3.1', '3.14', '3.142', '3.1416', '3.14159']

```

## 6.1.4. Listas por comprensión anidadas

La expresión inicial de una comprensión de listas puede ser cualquier expresión arbitraria, incluyendo otra comprensión de listas.

Considera el siguiente ejemplo de una matriz de 3x4 implementada como una lista de tres listas de largo 4:

```

>>> matriz = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]

```

La siguiente comprensión de lista transpondrá las filas y columnas:

```

>>> [[fila[i] for fila in matriz] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]

```

Como vimos en la sección anterior, la lista de comprensión anidada se evalúa en el contexto del **for** ([reference.html#for](#)) que lo sigue, por lo que este ejemplo equivale a:

```
>>> transpuesta = []
>>> for i in range(4):
...     transpuesta.append([fila[i] for fila in matriz])
...
>>> transpuesta
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

el cual, a la vez, es lo mismo que:

```
>>> transpuesta = []
>>> for i in range(4):
...     # las siguientes 3 líneas hacen la comprensión de listas anidada
...     fila_transpuesta = []
...     for fila in matriz:
...         fila_transpuesta.append(fila[i])
...     transpuesta.append(fila_transpuesta)
...
>>> transpuesta
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

En el mundo real, deberías preferir funciones predefinidas a declaraciones con flujo complejo. La función **zip()** haría un buen trabajo para este caso de uso:

```
>>> list(zip(*matriz))
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

Ver *Desempaquetando una lista de argumentos* ([controlflow.html#tut-unpacking-arguments](http://controlflow.html#tut-unpacking-arguments)) para detalles en el asterisco de esta línea.

## 6.2. La instrucción **del** ([reference.html#del](http://reference.html#del))

Hay una manera de quitar un ítem de una lista dado su índice en lugar de su valor: la instrucción **del** ([reference.html#del](http://reference.html#del)). Esta es diferente del método **pop()**, el cual devuelve un valor. La instrucción **del** ([reference.html#del](http://reference.html#del)) también puede usarse para quitar secciones de una lista o vaciar la lista completa (lo que hacíamos antes asignando una lista vacía a la sección). Por ejemplo:

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
```

**del** ([reference.html#del](http://reference.html#del)) puede usarse también para eliminar variables:

```
>>> del a
```

Hacer referencia al nombre `a` de aquí en más es un error (al menos hasta que se le asigne otro valor). Veremos otros usos para **del** ([reference.html#del](http://reference.html#del)) más adelante.

## 6.3. Tuplas y secuencias

Vimos que las listas y cadenas tienen propiedades en común, como el indizado y las operaciones de seccionado. Estas son dos ejemplos de datos de tipo *secuencia* (ver *Tipos integrados* (<reference.html#typeseq>)). Como Python es un lenguaje en evolución, otros datos de tipo secuencia pueden agregarse. Existe otro dato de tipo secuencia estándar: la *tupla*.

Una tupla consiste de un número de valores separados por comas, por ejemplo:

```
>>> t = 12345, 54321, 'hola!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hola!')
>>> # Las tuplas pueden anidarse:
... u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hola!'), (1, 2, 3, 4, 5))
>>> # Las tuplas son inmutables:
... t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # pero pueden contener objetos mutables:
... v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
```

Como puedes ver, en la salida las tuplas siempre se encierran entre paréntesis, para que las tuplas anidadas puedan interpretarse correctamente; pueden ingresarse con o sin paréntesis, aunque a menudo los paréntesis son necesarios de todas formas (si la tupla es parte de una expresión más grande). No es posible asignar a los ítems individuales de una tupla, pero sin embargo sí se puede crear tuplas que contengan objetos mutables, como las listas.

A pesar de que las tuplas puedan parecerse a las listas, frecuentemente se utilizan en distintas situaciones y para distintos propósitos. Las tuplas son *inmutables* y normalmente contienen una secuencia heterogénea de elementos que son accedidos al desempaquetar (ver más adelante en esta sección) o indizar (o incluso acceder por atributo en el caso de las **namedtuples**). Las listas son *mutables*, y sus elementos son normalmente homogéneos y se acceden iterando a la lista.

Un problema particular es la construcción de tuplas que contengan 0 o 1 ítem: la sintaxis presenta algunas peculiaridades para estos casos. Las tuplas vacías se construyen mediante un par de paréntesis vacío; una tupla con un ítem se construye poniendo una coma a continuación del valor (no alcanza con encerrar un único valor entre paréntesis). Feo, pero efectivo. Por ejemplo:

```
>>> vacia = ()
>>> singleton = 'hola',    # <-- notar la coma al final
>>> len(vacia)
0
>>> len(singleton)
1
>>> singleton
('hola',)
```

La declaración `t = 12345, 54321, 'hola!'` es un ejemplo de *empaquetado de tuplas*: los valores 12345, 54321 y 'hola!' se empaquetan juntos en una tupla.

La operación inversa también es posible:

```
>>> x, y, z = t
```

Esto se llama, apropiadamente, *desempaquetado de secuencias*, y funciona para cualquier secuencia en el lado derecho del igual. El desempaquetado de secuencias requiere que la cantidad de variables a la izquierda del signo igual sea el tamaño de la secuencia. Notá que la asignación múltiple es en realidad sólo una combinación de empaquetado de tuplas y desempaquetado de secuencias.

## 6.4. Conjuntos

Python también incluye un tipo de dato para *conjuntos*. Un conjunto es una colección no ordenada y sin elementos repetidos. Los usos básicos de éstos incluyen verificación de pertenencia y eliminación de entradas duplicadas. Los conjuntos también soportan operaciones matemáticas como la unión, intersección, diferencia, y diferencia simétrica.

Las llaves o la función `set()` pueden usarse para crear conjuntos. Notá que para crear un conjunto vacío tenés que usar `set()`, no `{}`; esto último crea un diccionario vacío, una estructura de datos que discutiremos en la sección siguiente.

Una pequeña demostración:

```
>>> canasta = {'manzana', 'naranja', 'manzana', 'pera', 'naranja', 'banana'}
>>> print fruta          # muestra que se removieron los duplicados
{'pera', 'manzana', 'banana', 'naranja'}
>>> 'naranja' in canasta  # verificación de pertenencia rápida
True
>>> 'yerba' in canasta
False

>>> # veamos las operaciones para las letras únicas de dos palabras
...
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                # letras únicas en a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                            # letras en a pero no en b
{'r', 'b', 'd'}
>>> a | b                            # letras en a o en b o en ambas
{'a', 'c', 'b', 'd', 'm', 'l', 'r', 'z'}
>>> a & b                            # letras en a y en b
{'a', 'c'}
>>> a ^ b                            # letras en a o b pero no en ambos
{'b', 'd', 'm', 'l', 'r', 'z'}
```

De forma similar a las *comprensiones de listas*, está también soportada la comprensión de conjuntos:

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
```

## 6.5. Diccionarios



Otro tipo de dato útil incluido en Python es el *diccionario* (ver *Tipos integrados* (<reference.html#typesmapping>)). Los diccionarios se encuentran a veces en otros lenguajes como “memorias asociativas” o “arreglos asociativos”. A diferencia de las secuencias, que se indexan mediante un rango numérico, los diccionarios se indexan con *claves*, que pueden ser cualquier tipo inmutable; las cadenas y números siempre pueden ser claves. Las tuplas pueden usarse como claves si solamente contienen cadenas, números o tuplas; si una tupla contiene cualquier objeto mutable directa o indirectamente, no puede usarse como clave. No podés usar listas como claves, ya que las listas pueden modificarse usando asignación por índice, asignación por sección, o métodos como **append()** y **extend()**.

Lo mejor es pensar en un diccionario como un conjunto no ordenado de pares *clave: valor*, con el requerimiento de que las claves sean únicas (dentro de un diccionario en particular). Un par de llaves crean un diccionario vacío: `{}`. Colocar una lista de pares clave:valor separados por comas entre las llaves añade pares clave:valor iniciales al diccionario; esta también es la forma en que los diccionarios se presentan en la salida.

Las operaciones principales sobre un diccionario son guardar un valor con una clave y extraer ese valor dada la clave. También es posible borrar un par clave:valor con `del`. Si usás una clave que ya está en uso para guardar un valor, el valor que estaba asociado con esa clave se pierde. Es un error extraer un valor usando una clave no existente.

Hacer `list(d.keys())` en un diccionario devuelve una lista de todas las claves usadas en el diccionario, en un orden arbitrario (si las querés ordenadas, usá en cambio `sorted(d.keys())`).<sup>[2]</sup> Para controlar si una clave está en el diccionario, usá el `in` (<reference.html#in>).

Un pequeño ejemplo de uso de un diccionario:

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'sape': 4139, 'jack': 4098, 'guido': 4127}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'irv': 4127, 'guido': 4127}
>>> list(tel.keys())
['irv', 'guido', 'jack']
>>> sorted(tel.keys())
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

El constructor **dict()** crea un diccionario directamente desde secuencias de pares clave-valor:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'jack': 4098, 'guido': 4127}
```

Además, las comprensiones de diccionarios se pueden usar para crear diccionarios desde expresiones arbitrarias de clave y valor:

```
>>> {x: x ** 2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

Cuando las claves son cadenas simples, a veces resulta más fácil especificar los pares usando argumentos por palabra clave:

```
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'jack': 4098, 'guido': 4127}
```

## 6.6. Técnicas de iteración

Cuando iteramos sobre diccionarios, se pueden obtener al mismo tiempo la clave y su valor correspondiente usando el método **items()**.

```
>>> caballeros = {'gallahad': 'el puro', 'robin': 'el valiente'}
>>> for k, v in caballeros.items():
...     print(k, v)
...
gallahad el puro
robin el valiente
```

Cuando se itera sobre una secuencia, se puede obtener el índice de posición junto a su valor correspondiente usando la función **enumerate()**.

```
>>> for i, v in enumerate(['ta', 'te', 'ti']):
...     print(i, v)
...
0 ta
1 te
2 ti
```

Para iterar sobre dos o más secuencias al mismo tiempo, los valores pueden emparejarse con la función **zip()**.

```
>>> preguntas = ['nombre', 'objetivo', 'color favorito']
>>> respuestas = ['lancelot', 'el santo grial', 'azul']
>>> for p, r in zip(preguntas, respuestas):
...     print('Cual es tu {0}? {1}.'.format(p, r))
...
Cual es tu nombre? lancelot.
Cual es tu objetivo? el santo grial.
Cual es tu color favorito? azul.
```

Para iterar sobre una secuencia en orden inverso, se especifica primero la secuencia al derecho y luego se llama a la función **reversed()**.

```
>>> for i in reversed(range(1, 10, 2)):
...     print(i)
...
9
7
5
3
1
```

Para iterar sobre una secuencia ordenada, se utiliza la función **sorted()** la cual devuelve una nueva lista ordenada dejando a la original intacta.

```
>>> canasta = ['manzana', 'naranja', 'manzana', 'pera', 'naranja', 'banana']
>>> for f in sorted(set(canasta)):
...     print(f)
...
banana
manzana
naranja
pera
```

A veces uno intenta cambiar una lista mientras la está iterando; sin embargo, a menudo es más simple y seguro crear una nueva lista:

```
>>> datos = [56.2, float('NaN'), 51.7, 55.3, 52.5, float('NaN'), 47.8]
>>> datos_filtrados = []
>>> for valor in datos:
...     if not math.isnan(valor):
...         datos_filtrados.append(valor)
...
>>> datos_filtrados
[56.2, 51.7, 55.3, 52.5, 47.8]
```

## 6.7. Más acerca de condiciones

Las condiciones usadas en las instrucciones `while` e `if` pueden contener cualquier operador, no sólo comparaciones.

Los operadores de comparación `in` y `not in` verifican si un valor está (o no está) en una secuencia. Los operadores `is` e `is not` comparan si dos objetos son realmente el mismo objeto; esto es significativo sólo para objetos mutables como las listas. Todos los operadores de comparación tienen la misma prioridad, la cual es menor que la de todos los operadores numéricos.

Las comparaciones pueden encadenarse. Por ejemplo, `a < b == c` verifica si `a` es menor que `b` y además si `b` es igual a `c`.

Las comparaciones pueden combinarse mediante los operadores booleanos `and` y `or`, y el resultado de una comparación (o de cualquier otra expresión booleana) puede negarse con `not`. Estos tienen prioridades menores que los operadores de comparación; entre ellos `not` tiene la mayor prioridad y `or` la menor, o sea que `A and not B or C` equivale a `(A and (not B)) or C`. Como siempre, los paréntesis pueden usarse para expresar la composición deseada.

Los operadores booleanos `and` y `or` son los llamados operadores *cortocircuito*: sus argumentos se evalúan de izquierda a derecha, y la evaluación se detiene en el momento en que se determina su resultado. Por ejemplo, si `A` y `C` son verdaderas pero `B` es falsa, en `A and B and C` no se evalúa la expresión `C`. Cuando se usa como un valor general y no como un booleano, el valor devuelto de un operador cortocircuito es el último argumento evaluado.

Es posible asignar el resultado de una comparación u otra expresión booleana a una variable. Por ejemplo,

```
>>> cadena1, cadena2, cadena3 = '', 'Trondheim', 'Paso Hammer'
>>> non_nulo = cadena1 or cadena2 or cadena3
>>> non_nulo
'Trondheim'
```

Notá que en Python, a diferencia de C, la asignación no puede ocurrir dentro de expresiones. Los programadores de C pueden renegar por esto, pero es algo que evita un tipo de problema común encontrado en programas en C: escribir `=` en una expresión cuando lo que se quiere escribir es `==`.

## 6.8. Comparando secuencias y otros tipos

Las secuencias pueden compararse con otros objetos del mismo tipo de secuencia. La comparación usa orden *lexicográfico*: primero se comparan los dos primeros ítems, si son diferentes esto ya determina el resultado de la comparación; si son iguales, se comparan los siguientes dos ítems, y así sucesivamente hasta llegar al final de alguna de las secuencias. Si dos ítems a comparar son ambos secuencias del mismo tipo, la comparación lexicográfica es recursiva. Si todos los ítems de dos secuencias resultan iguales, se considera que las secuencias son iguales.

Si una secuencia es una subsecuencia inicial de la otra, la secuencia más corta es la menor. El orden lexicográfico para cadenas de caracteres utiliza el orden de códigos Unicode para caracteres individuales. Algunos ejemplos de comparaciones entre secuencias del mismo tipo:

```
(1, 2, 3) < (1, 2, 4)
[1, 2, 3] < [1, 2, 4]
'ABC' < 'C' < 'Pascal' < 'Python'
(1, 2, 3, 4) < (1, 2, 4)
(1, 2) < (1, 2, -1)
(1, 2, 3) == (1.0, 2.0, 3.0)
(1, 2, ('aa', 'ab')) < (1, 2, ('abc', 'a'), 4)
```

Observá que comparar objetos de diferentes tipos con `<` o `>` es legal siempre y cuando los objetos tengan los métodos de comparación apropiados. Por ejemplo, los tipos de números mezclados son comparados de acuerdo a su valor numérico, o sea 0 es igual a 0.0, etc. Si no es el caso, en lugar de proveer un ordenamiento arbitrario, el intérprete generará una excepción **TypeError**.

### Footnotes

- [1] Otros lenguajes pueden devolver el objeto mutado, lo cual permite encadenado de métodos, como `d->insert("a")->remove("b")->sort();`.
- [2] Llamar a `d.keys()` devolverá un objeto *vista de diccionario*. Soporta operaciones como prueba de pertenencia e iteración, pero sus contenidos dependen del diccionario original – son sólo una *vista*.